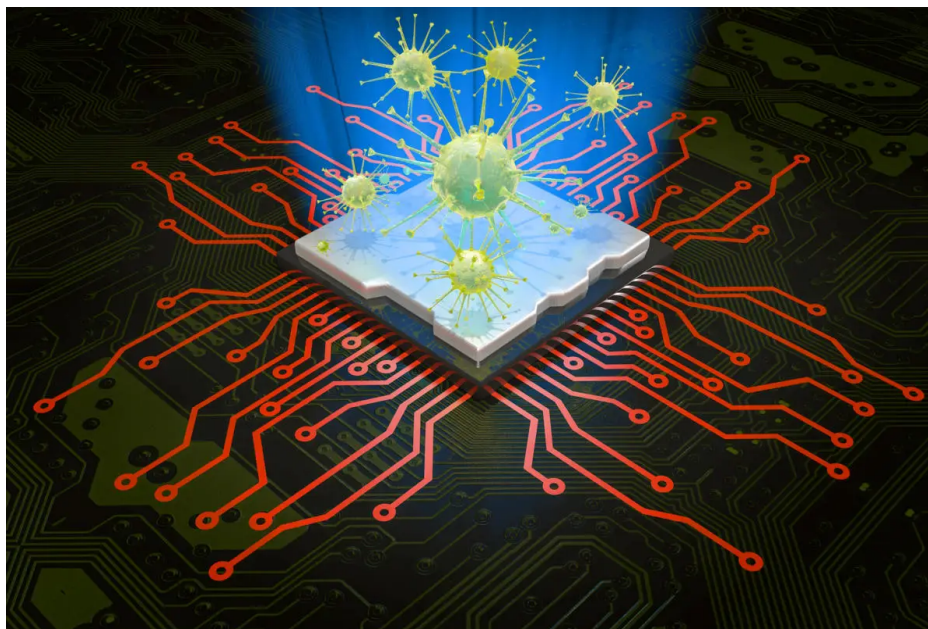


INSuRE Spring 2022
Final Report - May 4, 2022
How It's Made: Malware Edition
Sponsor: MITRE
Technical Director: Frank Posluszny - malwarefrank@mitre.org



University of Arizona and University of Texas at Dallas
Team: Binary Sunrise

Students Investigators

Abhijeet Dabas - Graduate Third Semester - abhid@email.arizona.edu
Jacob Staggs - Graduate Second Semester - jastaggs@email.arizona.edu
Timothy Colvert - Graduate Second Semester - tcolvert@email.arizona.edu
Tri Nguyen - Undergraduate Sixth Semester - txn180008@utdallas.edu

Faculty Support

Sharon O'Neal - sharononeal@arizona.edu
William Hayes - billhayes@arizona.edu
Bhavani Thuraisingham - bhavani.thuraisingham@utdallas.edu
Brian Ricks - absolutejunk@utdallas.edu

Table of Contents

1.0 Executive Summary	2
2.0 Introduction	2
2.1 Top Level Project Description	2
2.2 Purpose Statement	3
2.3 Motivation	3
3.0 Literature Review	4
3.1 Motivation	4
3.2 Existing Research on Malware Characteristics and Analysis	4
3.3 Justification for Current Plan	8
4.0 Methods and procedures	8
4.1 Characterization of Methods	8
4.2 Procedures and Plan	9
4.3 Schedule	11
4.4 Deliverables	11
4.5 Limitations and Delimitations	13
5.0 Findings	13
5.1 Findings Overview	13
5.2 Detailed Findings	14
6.0 Issues	28
7.0 Conclusions and Recommendations	31
8.0 Future Work	31
9.0 References	32
10.0 Team Biographies / Tasking	33
11.0 Appendix A: Acronyms	36
12.0 Appendix B: Terms and Definitions	37

1.0 Executive Summary

Given the difficulties involved in all stages of malware analysis, a common research topic in the industry is how to improve the efficacy and accuracy of malware analysis. Recent studies and industry movement have shown that toolmarks are incredibly useful for gaining a complete picture of the function and goals of a sample (HexRays, 2011; Rahimian et al., 2015; McVey, 2015; Alrabee, 2019). Specifically, the development of the Malware Development Life Cycle (MDLC) shows that software and malware tool marks can be fused together to gain an idea of the attacker's mindset, visions, and goals in developing a specific malware sample (Posluszny, 2020). This project aimed to propose a malware analysis tool that employs toolmark analysis, such as compiler provenance and static library code identification, and the MDLC to create a new harmony of cyber threat intelligence and malware analysis that, to date, has not been seen within the industry. This research project has considered and incorporated previous research and work done on this topic in order to inform the process of malware characterization. After developing several ground truth malware samples and analyzing their development within the existing Malware Development Life Cycle (MDLC) framework, the team has gained considerable insight into the possible decisions that go into the creation of a remote access malware, including the reliance on static libraries API function calls, as well as insights into the discrepancy between what information an analyst can gather from a disassembled malware and the true characterizations of the malware as designed by the attacker.

From these insights, the Binary Sunrise team recommends that a future team build a Python-based malware analysis tool that collects and explains information about malware using the MDLC framework. The tool would make extensive use of existing toolchains and datasets to detect the decisions made in each phase of the development of a given malware sample and create a human-readable report similar to how the ground truth samples created in this project were analyzed.

2.0 Introduction

2.1 Top Level Project Description

Overall, this project aims to propose a new malware analysis tool that simplifies the analysis process by finding and digesting information endemic to all malware samples, such as compiler toolmarks, library functions, and code transforms. Designing such a tool will require a new line of thought in analyzing malware samples. Specifically, the team will integrate our findings and existing techniques with MITRE's Malware Development Life Cycle (MDLC) to formulate a framework by which a new malware analysis tool class could be implemented. By conducting research to understand how information artifacts specific to various toolchains and design decisions impact the analysis process, the team proposed a new tool idea that could be both a massive boon to analysis efficacy and advance the industry's understanding of these information artifacts and the characterization methods necessary to exploit the artifacts.

2.2 Purpose Statement

The purpose of this project was to research malware analysis concepts and frameworks in order to facilitate the proposal of a new malware analysis tool employing often neglected malware analysis topics. A secondary purpose was to create more ground truth malware samples for future INSuRE teams to employ in extensions of this project. This project also aims to change the current state of affairs by helping to provide sample characteristics of these ground truth malware samples within the MDLC, where each sample will be broken down and analyzed in the context of the MDLC. To date, few full analyses of malware samples have been performed using the MDLC. In short, malware analysis is a difficult task and all stages of the process are slow to complete. The current practices do not always emphasize the importance of understanding the attack and his or her goals in creating a malware package. This project provided some insight into how the overall malware analysis process can be improved.

2.3 Motivation

Malware analysis is time consuming and the flow of the analysis process is heavily based on a variety of factors, such as tools used to create the malware sample, programmer thought process and decision models, malware behavior, and more (McVey, 2015). A current major research focus in the malware analysis field is figuring out a method by which the toolmarks, code transforms, and packing of malware samples can be exploited informationally to aid in the analysis process. Consider an analyst that is conducting advanced static analysis in IDAPro on a major malware sample. The sample is complex; it has dozens of subroutines that are called repeatedly in numerous overarching routines. In IDAPro, these subroutines were not detected automatically by the tool's built-in compiler detection methods. As such, the analyst will need to examine these subroutines manually to determine their collective functionality, thus depriving the analyst of valuable time that could be better spent reverse engineering more demanding portions of the sample, such as obfuscated sections. Had the analyst possessed a tool capable of determining the compiler used to construct the malware, and thus compiler-related functions, the reverse-engineering workload could have been significantly reduced (Rahimian et al., 2015). With compiler and toolchain provenance determination techniques, various useful information points, such as compiler type, version, and optimization levels, can be determined and applied to the malware analysis process (Rahimian et al., 2015). With the illustration of the previous example, it is clear why researching provenance and other toolmarks in respect to malware analysis is an important and potentially lucrative venture. Mapping these artifacts into the MDLC will help an analyst produce a more trustworthy report in a more efficient manner. Given that the MDLC and cyber threat intelligence as it relates to malware analysis is an underutilized resource, this endeavor was worthy. To the Binary Sunrise team, it was essential to bring awareness to the value of toolmark analysis and the MDLC in order to revolutionize the malware analysis industry.

3.0 Literature Review

3.1 Motivation

The team's final product explores areas of malware analysis that have conventionally received little attention. Information about the MDLC itself is not available online in any form. Ultimately, the team aspired to research and promote a new line of thinking regarding malware analysis. Specifically, the final product brought attention to possibilities in developing commercial methods to more effectively employ and digest toolmark information such as compiler provenance and encourage analysts to begin reverse engineering malware samples with an attacker's mindset by framing analysis results using the MDLC, which is inherently attacker-oriented.

The team wished to propose an implementation model for a theoretical tool that can identify toolmarks, utilizing either novel methods developed by the team or already existing analysis solutions, and correlate these toolmarks to the MDLC. As such, it was essential for the team to research and identify software toolmarks. With this research, the team found which toolmark analysis processes would allow the proposed tool to lend itself to performing more efficient and thorough analysis of malicious binaries. Given the team's goal to create ground truth malware samples, the team also devoted research time to exploring the purpose of ground truth samples and the most appropriate capabilities to implement. This was essential for beginning the development process for the ground truth samples and was thus a high priority for the team.

3.2 Existing Research on Malware Characteristics and Analysis

Toolmarks and Compiler Provenance

All software, and by extension malware, have toolmarks that are left behind during the coding and compilation process. Broadly, these toolmarks can be divided into three categories: code format and structure, compiler-specific information, the programmer's choices. In any malware samples, toolmarks will be spread throughout the binary in these three forms. Some common toolmarks found in Windows malware samples are Portable Executable (PE) metadata, such as packing information and build information including language settings and build date, and function call information, including static library function calls (McVey, 2015).

Extensive research has been performed on correlating both malware and commercial software toolmarks to useful information for reverse engineering. One major research area is compiler provenance. In essence, compiler provenance refers to the extraction and analysis of compiler toolmarks, such as compiler family, version, and functions, in any given piece of software. A 2015 paper by Concordia University, *BinComp: A stratified approach to compiler provenance attribution*, details both the history of compiler provenance efforts and more contemporary initiatives. In 2010 and 2011, a series of pioneering studies were conducted on extracting compiler information, labeling functions found in binaries stripped of symbols, identifying compilers used to build stripped binaries, and finally establishing toolchain attribution for stripped binaries. While these studies were useful, their results were made less

useful for practical usage by their small and restrictive data sets (Rahimian et al., 2015). These issues were addressed with the revolutionary implementation of BinComp by the Concordia University research team.

BinComp is a three-layered machine-learning approach to extracting compiler provenance information from unobfuscated C++ binaries. Using advanced machine-learning techniques and a large set of sample programs to delve into compiler function information, BinComp was able to improve the accuracy of previous provenance analysis implementations and granularity of results. BinComp was revolutionary because it achieved record high accuracy in determining compiler family, version, and optimization levels and thus proved that accurate automated compiler provenance is possible and achievable. However, it was limited to unstripped binaries with no obfuscation; it left the determination of methods to deal with obfuscation and stripped binaries to later research (Rahimian et al., 2015). Unfortunately, this problem seems to persist into today's research; there is little information available on compiler provenance as it relates to stripped and obfuscated binaries.

Another major area related to both toolmarks and compiler provenance is the identification of statically-linked functions in malware samples. A statically-linked function is a function that is included in the compilation process by either a code library used by a code author or by the compiler itself to build the code. Understanding how the compiler adds functions to a binary at compile-time is essential to reverse engineering efforts. HexRays, the developer of the esteemed reverse engineering tool IDAPro, estimates that a medium-sized Windows executable employs anywhere from 1000 to 2500 library function calls (HexRays, 2011). In the 2019 report *On the feasibility of binary authorship characterization*, Alrabee, Debbabi, and Wang posited that common malware families, such as Ramnit and Lollipop, contained anywhere from 1600 to 8000 functions but composed of anywhere from 10 to 30 percent library and compiler functions. By automating the identification of these functions in any given malware sample, analysts stand to both drastically reduce their workload and increase their base understanding of a sample.

Extensive research exists regarding the identification of statically-linked library functions. Commercial malware analysis and reverse engineering tools, such as IDAPro, already implement signature matching that can accurately and quickly find common library functions, such as Windows API library function calls. In fact, these functionalities are already well-known and well-developed; they form the cornerstone of advanced static analysis (HexRays, 2011; McVey, 2015; Alrabee, 2019). Clearly, it is not necessary nor feasible to attempt to expand upon the identification of statically-linked library code. There is noticeably less information regarding the identification of compiler functions. Some already discussed research, namely BinComp, exists regarding the identification of compiler functions. BinComp is capable of finding and naming, with high confidence, hundreds of compiler functions from the most popular C++ compilers (Rahimian et al., 2015; Alrabee, 2019). Unfortunately, there are major gaps in research in compiler function identification. Very few resources exist on the public Internet that fully flesh out the topic of compiler function information; no true opposing ideas exist in this sphere. Other

research areas included Nucleus, which only functions to find function barriers and was thus not useful for the purposes of the team's project (Andriess et al, 2017).

MDLC

The MDLC is a methodology for identifying and categorizing the toolmarks left on a malware sample and correlating them to a malware author's thoughts and actions during development. This correlation forms the means by which an analyst can better understand the attacker's intentions and choices. To facilitate this process, the MDLC is split into eight steps, six of which are from the attacker's point of view and two of which are from the analyst's point of view (Posluszny, 2020).

The attacker side of the MDLC comprises the requirements, design, code, build, configure, and package phases. The requirements step occurs when the attacker or the attacker's employer posts a set of malware requirements. These requirements are stated at a high-level and could include potential malicious actions and operational security features. The design phase starts when the attacker begins designing the malware itself. Design artifacts may include algorithm decision and usage information, as well as the design and reverse engineering of any custom encryption or obfuscation schemes. The code phase begins when the attacker codes and implements the malware. Applicable code artifacts include programming language choice, use of specific libraries, and use of standard encryption libraries. The build phase occurs when the attacker builds the code, for example by compiling it. The chief build phase artifacts are compiler provenance information, as discussed previously. The configure stage occurs in pieces of malware that include configuration options for the end user, such as commercial remote access tools (RAT). Configure stage artifacts are more advanced features to identify, but can include configuration blocks, such as settings menus. The final stage, packaging, occurs when an attacker readies the malware for deployment. This happens, for example, when an attacker embeds malware in an exploitable document for distribution. Packaging artifacts include any packers, such as UPX, used to obfuscate a malware sample, as well as shellcode used to deploy malware (Posluszny, 2020). Correlating information artifacts gleaned from malware samples to these steps allows analysts to gain a much stronger understanding of the attacker's visions and goals. An informed analyst is a far more efficient one. Thus, the MDLC is an incredibly useful framework for gaining insight into the operations and goals of any given malware sample.

The two analyst-side steps of the MDLC are where all of the analyst's action's occur. First, the malware sample is collected in the course of an investigation. Next, the analysis phase is started. The analyst progresses through the steps of malware analysis, static and dynamic analysis, to gain information artifacts. During analysis, the analyst constantly reviews and maps artifacts to the MDLC. As these mappings occur, a clearer view of the attacker's mindset is gained, strengthening the results and output of the analysis (Posluszny, 2020).

Unfortunately, the MDLC is an underutilized framework; it is not formally published and is not taught at any universities. However, in its vision to link malware analysis and cyber threat intelligence, it is indispensable. Thankfully, the MDLC has been used in previous unpublished

projects. For example, the Spring 2020 UTD INSuRE team developed a malware analysis tool in Python that automatically extracts features from the Windows executable malware based on the last four stages of the MDLC. Among the features extracted were programming language, target architecture, cryptographic functions if any, packers, employed libraries, and more. However, their final output did not reflect any consideration of the MDLC, even though their final report mentioned it (Agarwal et al., 2020).

Ground Truth

Any project that attempts to build a statistical model of existing information requires the gathering of ground truth. Ground truth is the set of facts that are known to be true by direct measurement, as opposed to facts that are inferred from existing measurements. In the context of malware analysis, establishing ground truth requires that a team or analyst gathers a collection of malware whose entire development life cycle is known, including all elements of the requirements, design, code, build, configure, and package phases. Ground truth is important in verifying the correctness of the model of analysis concerning any gathered information artifacts. Since it would be infeasible to collect the full development life cycle of real malware from their developers, the most practicable means of obtaining ground truth for malware analysis is for analysts and researchers to develop their own malware from scratch and document their full development using the MDLC framework.

In the Spring 2020 semester, the UTD team developed their own data exfiltration tools that act as malware to establish ground truth for their malware analysis tool (Agarwal et al., 2020). The tools can extract, obfuscate, and send over HTTP files from an infected machine. They were packaged as standalone Windows executables to mimic the typical conditions of malware analysis, which is often done on self-contained executable malware. Finally, a list of features from the phases of the MDLC was created for each data exfiltration tool and used this data to verify the correctness of their malware analysis tool. Their results were valid, and their tool's output correct, because they created this ground truth. Had they not begun development with exactly-known data, then their models could have been questioned. The Binary Sunrise team managed to develop four ground truth samples; their properties will be discussed later.

Malware Capabilities

Our research narrowed down six main techniques in use by major threat actors, such as China's APT1 and Double Dragon, also known as APT41 (Alrawi et al., 2021; FireEye, 2013; FireEye, 2019). The first capability was file exfiltration, which aims to steal files from an infected system and then send those stolen files to the attacker. The second is dropping, which makes use of a so-called first stage malware payload to deliver another payload. The third feature is keylogging, in which a malicious actor or piece of software logs the keystrokes of a user. The fourth feature was persistence, a method by which malware will survive when a system reboots or the system powers down; persistent malware is programmed to reactivate after some specified action or system event. The fifth capability was anti-analysis and evasion techniques that are a

malware sample's attempt through various methods to evade detection and malware analysis, hampering the malware analysis activities. The sixth feature is command and control (C2), in which an attacker issues commands to the malware from an interface, such as a remote shell or bot controller. Understanding these six techniques was essential for the team set out to develop and implement ground truth samples for the final product. They served as the initial requirements negotiation material, which led to the overall requirements for the team's ground truth samples being established.

3.3 Justification for Current Plan

Ground Truth

The ground truth samples are of the utmost importance to the analysis and without the ground truth samples there is no way to continue in producing an accurate and efficient tool. The importance of ground truth in checking the validity of the model of analysis in relation to any acquired information artifacts cannot be overstated. Because collecting the whole development life cycle of genuine malware from its authors would be impossible, the most practical method of acquiring ground truth for malware analysis is for analysts and researchers to create their own malware from the beginning and record it using the MDLC framework.

4.0 Methods and Procedures

4.1 Characterization of Methods

Exploratory Research

The team began the project with exploratory research. The TD's initial research papers proved useful for introducing the team to the idea of toolmark analysis. Additional research into the sheer scope of malware proliferation in current time began to also familiarize the team with the overall problem. Further research on the MDLC assisted the team in framing the problem at hand regarding the characterization of malware, or lack thereof, in the current malware analysis industry. Exploratory research was revisited whenever the team encountered issues with our lines of thinking, for example, if we found information that contradicted our views. Extra exploratory research allowed the team to fully understand the issues at hand and completely dispel any informational conflicts; this was essential to the success of the project.

Constructive Research

Initial literature review was significantly helpful here in finding that compiler provenance and statically-linked library code research helped begin to illuminate solutions for malware sample processing. Along with finding solutions for malware sample processing, reviewing reverse engineering solutions and other malware analysis tools helped the team find concrete implementation ideas for a proposed malware analysis tool. Research into ground truth definitions and previous project requirements helped us start to imagine how to implement our ground truth samples and aided us in further research. Constructive research was also conducted during the development of ground truth samples. Each member conducted research that provided

tangible malware implementation details for different programming languages. This constructive implementation research took quite some time, but was necessary to create the project's ground truth.

Empirical Research

Through empirical research the team was able to further research into current state-of-the-art compiler provenance solutions that began to reveal to the team that we do not have the resources to employ any advanced compiler provenance techniques. The research revealed that it was far too advanced to replicate and we failed to have the timeframe required to obtain previous research teams' programs. This led the team to the decision to employ only basic compiler provenance techniques.

Research into reverse engineering tools and the MDLC showed the team that pursuing information about statically-linked library code and other code toolmarks would be significantly more fruitful than pursuing advanced compiler provenance detection methods. The team was able to identify workable tools for a final tool proposal and the team was able to identify useful statically-linked library code signature databases.

4.2 Procedures and Plan

Our research project plan used a three phase approach. The first phase of the project entailed researching current solutions to determine compiler provenance information and statically-linked function information, current major malware techniques to implement and toolchains to employ when creating our ground truth test samples, appropriate code-sharing and test environments to employ, and whether or not it would be more appropriate to develop a standalone tool or a plugin for existing tools. The end result of this phase was a course of action to complete the next three phases. Phase Two was the group development of the test samples that will be fed to the team's final project. The team was subdivided into two teams to develop ground truth samples in different programming languages at the low-level and high-level standard. The team analyzed the results of phase one's research and plan to make decisions on how many samples to code, what tools to employ, and what malware features to include. Phase Three consisted of the team presenting a proposed malware analysis tool and how it will provide a framework for how the MDLC can be examined.

Originally our team had a four-phased approach that was reduced to our current three-phased approach due to our time constraints thus not allowing the team to implement the full-scope malware analysis tool. The team and technical director (TD) agreed that a proposed tool based on ground truth sample analysis and toolchain research would be appropriate.

- 1st Phase - Conduct Research
- 2nd Phase - Implement Ground Truth Samples
- 3rd Phase - Propose Malware Analysis Tool

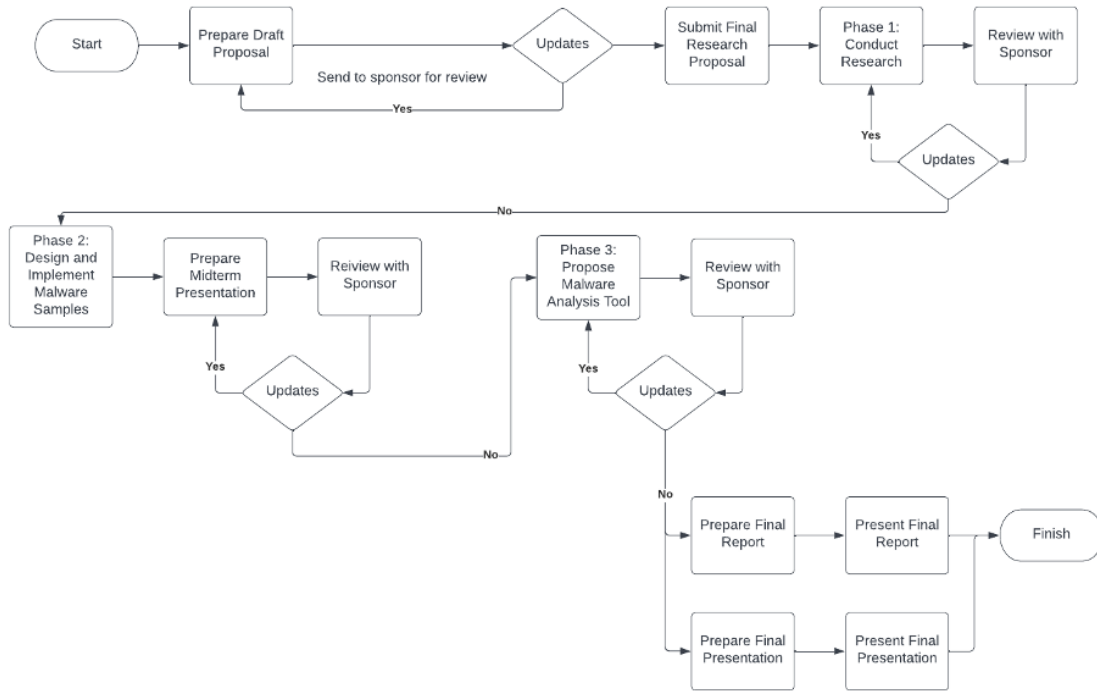


Figure 1. This is the flowchart defining how the team progressed through the project phases.

4.3 Schedule

Phase One Milestones		Completed as Expected							
Name		Status	Completion Date						
Research Compiler Provenance Solutions		Complete	February 27, 2022						
Research Function Identification Solutions		Complete	February 27, 2022						
Research the MITRE MDLC		Complete	February 27, 2022						
Research Malware Capabilities		Complete	February 27, 2022						
Phase Two Milestones		Completed as Expected							
Name		Status	Completion Date	Notes					
Obtain Ground Truth Sample Requirements from TD		Complete	February 25, 2022	N/A					
Research Languages to Employ in Samples		Complete	March 1, 2022	N/A					
Research Implementation of Malware Capabilities		Complete	April 14, 2022	Implementation Information is Language-Specific and mostly Non-Academic in Nature					
Implement Malware Samples		Complete	April 28, 2022	Implementation was much more difficult than anticipated; samples mostly done					
Phase Three Milestones		Phase Three has been restructured; scope reduction as prescribed by team TD; Completed with Scope Reductions							
Name		Status	Completion Date	Notes					
Research Potential Toolchains for Proposal		Complete	April 15, 2022	Needed to research viable toolchains for tool proposal; mainly constructive and empirical research					
Formulate MDLC Data Structures		Complete	April 28, 2022	Need design data structures for representing the MDLC in a computer program; language-agnostic					
Design Tool Execution Flow and Output Processing		Complete	April 28, 2022	Describe how the toolchains and MDLC will be integrated and how the sample will be processed					
Complete Malware Analysis Tool Proposal		Complete	April 28, 2022	Blend together all products of the revised phase into a coherent proposal					
Phase Four Milestones		Phase Four has been eliminated; scope reduction as prescribed by team TD							
INSuRE Tasks: Current Expected Completion Dates and Listed Due Dates									
Deliverable/Milestone	Expected Completion Date		Due Date (If Applicable)	Status	Notes				
Project Proposal	February 4, 2022		February 4, 2022	Complete					
Midterm Presentation Submission	March 24, 2022		March 24, 2022	Complete					
Live Midterm Presentation	March 25, 2022		March 25, 2022	Complete	This will be performed live; it is a presentation				
Final Presentation Submission	April 28, 2022		April 28, 2022	Complete					
Live Final Report Presentation	April 29, 2022		April 29, 2022	Complete	This will be performed live; it is a presentation				
INSuRE Research Experience/Less	May 3, 2022		May 4, 2022	Complete					
Major Project Deliverables									
Name	Description					Due Date	Status		
Four Ground Truth Samples	Malware samples used to test correctness and efficacy of main product; reduced from six to four for time reasons					April 28, 2022	Complete		
One Proposed Malware Analysis Tool Proposal	To Proposal for Malware analysis tool using research findings to determine best strides for determining malware functionality and engineering mindset					April 28, 2022	Complete		

Figure 2. A spreadsheet that details deliverables, their completion status and dates, and any important notes about them.

Figure two depicts our project schedule, as it was ultimately completed during the semester. Phase 1 and its subtasks were completed by February 27, 2022. Phase 2 and its milestones were completed by April 28, which was much later than originally expected. Phase 3, in its reduced scope, was completed at the same time.

4.4 Deliverables

The following tables list all applicable due dates for both University of Arizona-specific assignments and INSuRE tasks. Each task was completed the day it was due, or a single day before. All completed tasks were within given quality specifications.

Expected Completion Dates and Listed INSuRE Due Dates

Deliverables/Milestone	Expected Completion Date	Due Date
Team Assignments	January 12, 2022	January 12, 2022
Bids	January 18, 2022	January 18, 2022
Proposals	February 4, 2022	February 4, 2022

Final Midterm Presentation	March 15, 2022	March 17, 2022
Final Report Presentation	April 26, 2022	April 28, 2022
Final Report	May 2, 2022	May 4, 2022
INSuRE Research Experience and Lessons Learned	May 3, 2022	May 4, 2022

Complete Due Date List for University Assignments

Deliverables	Listed Due Date	Status
Bids	January 18, 2022	Completed
Project Proposal	February 4, 2022	Completed
Midterm Progress Presentation Outline	March 4, 2022	Completed
Draft Midterm Presentation	March 14, 2022	Completed
Final Midterm Status Report Presentation	March 17, 2022	Completed
Outline for Final Report	April 10, 2022	Completed
Draft Final Report Presentation	April 17, 2022	Completed
Draft Final Report	April 24, 2022	Completed
Final Report Presentation	April 28, 2022	Completed
Final Report	May 4, 2022	Completed
INSuRE Research Experience and Lessons Learned	May 4, 2022	Completed

4.5 Limitations and Delimitations

Limitations

There have been a variety of limitations throughout the entirety of the project. Starting with Advanced Compiler Provenance tools not being easily available. There are no commercially available compiler provenance tools that exist past basic tools without the ability to read compiler functions. This meant the team was unable to obtain an advanced compiler provenance tool which influenced a later mentioned delimitation. The team struggled in securing a cloud virtualization service, one attempt made through the University of Arizona was found to not allow any form of malware testing. This gave the team the challenge of setting up their own test beds resulting in a significant loss of time on the project and reduced the ability to collaborate online. Lastly, the team found major issues with time constraints as this project was only a semester long and development of ground truth samples was a harder and more time consuming task than originally anticipated. This meant the team was forced to reduce their scope of the project as of April 15, 2022. This scope reduction eliminated the requirement to implement a malware analysis tool and instead changed it to proposing a malware analysis tool. Additionally, the number of required ground truth samples was reduced from six to four.

Delimitations

Over the course of the project, team Binary Sunrise implemented multiple delimitations. The team decided not to pursue the compiler provenance as in depth as originally intended. The project goal was to characterize malware design choices. The team ultimately found that advanced compiler provenance detection was not as important as the statically-linked library code research in this regard, especially considering the difficulties associated with advanced compiler provenance. The team also decided to limit the tool's targets to that of only Windows PE files due to the fact that it would be too difficult to process multiple dedicated tools for different languages and different architectures. This was found difficult due to needing different toolchains required to analyze different executable file formats. Only statically-linked library identification tools, such as Rizin, were cross-platform. Lastly, it was decided to employ tools to wrap high-level interpreted languages into Windows PE executables. An ideal scenario regarding high-level interpreted languages would be samples in these languages are wrapped using available utilities, such as PyInstaller and py2exe for Python and packr and Launch4j for Java.

5.0 Findings

5.1 Findings Overview

Our overall findings were limited by the aforementioned decrease in project scope. Overall, the team was able to produce four ground truth samples, two C++ and two Java samples. The final C++ sample composed of three modules, a main executable, and attacker interface that totalled in at over 2000 lines of code. The final Java sample was composed of a single monolithic file that was approximately 600 lines long. All of our produced samples met at least the bare

minimum functionality as required by the technical director's specifications. Furthermore, the team was able to characterize each of our produced samples within the context of the MDLC.

Ultimately, the development cycle of our ground truth samples proved to the team the inherent difficulty of implementing original malware. Even though our sample requirements were basic, the team took a majority of the semester to produce four functional malware samples. Truly, creating acceptable ground truth samples is an arduous task. The extensive research that the team undertook while designing the samples presented us with numerous difficult design decisions to make; there were potentially dozens of different APIs, functions, and data structures that could have been employed for each capability. Furthermore, once we made our design decisions, we often found that our previous decisions were infeasible to implement or did not work as we had envisioned. Moreover, we found that our previous classes had not prepared entirely for a project of this scale; we found ourselves frequently researching how to implement minute details that were glossed over in previous courses or having to research new techniques to build and test our ground truth samples. Finally, many of our development questions were obscure, and little to no information existed that could solve our specific issues that arose during the programming of our samples. This set of conditions contributed to a difficult development process.

The team was not able to build a malware analysis tool; we simply ran out of time to design and program a viable tool. Instead, the team reduced our project scope regarding our malware analysis tool to simplifying designing and outlining a potential solution. Our tool proposal outlines the characteristics of our tool, data structures necessary to complete the tool, a potential execution flow model for our tool, and suggested toolchains to include within the tool.

5.2 Detailed Findings

Ground Truth Sample Requirements

At the start of the project's second phase, Binary Sunrise's technical director issued an initial set of requirements for our ground truth samples. The team was to implement a remote access malware sample with the following capabilities: the sample must be able to list files and directories on the infect system; the attacker must be able to upload files from his or her system to the infected system; the attacker must be able to download files from the infected system to his or her system; the attacker must be able to execute arbitrary executables located on the infected system, with optional command-line arguments; and the attacker must be able to send commands to the infect system's default command shell. Additional initial requirements also included the obfuscation of network traffic based on a hardcoded text-based password, anti-debugging and anti-sandboxing techniques, and simple code or data obfuscation. Lastly, the technical director required that the team implement the ground truth sample in at least one high-level programming language and one low-level programming language.

Once the team neared completion of its initial malware samples, the technical director imposed an additional requirement for our final malware sample. The team was required to have infected systems provide the following information to the attacker upon connection: the current

user's username, the system's name, the system's Workgroup or Domain name, and the system's operating system version and name. Because the final sample requirements are only a simple addition to the first sample set's requirements, the team only presents characterizations of our final samples; including characterizations for mostly similar samples would be redundant.

Ground Truth Sample Development

At the beginning of the ground truth sample development process, the team divided itself into two subgroups, named Team 1 and Team 2. Team 1, which was composed of Jacob Staggs and Abhijeet Dabas, was tasked with creating the low-level ground truth samples. This subgroup decided to implement its samples in Microsoft Visual C++ and employed the Microsoft Visual C++ Compiler (MSVC) to build its code samples. Team 2, which included Tri Nguyen and Timothy Colvert, was assigned the task of creating the team's high level ground truth samples. Team 2 resolved to implement its samples in Java and employed Java's rich feature to build its samples after implementation.

Team 1 began its sample development in early March. It began development with an overview of the Windows API and its constituent libraries. It found that the WS2_32, MSWSOCK, KERNEL32, and ADVAPI32 would be useful developmental libraries. From mid-March to late-April, Team 1 struggled through its malware sample development. The team ran into dozens of issues across its three functionality modules, requiring hundreds of recompilations and several dozens of hours of debugging. Team 1 dedicated approximately 150 to 200 man hours to implementing the sample during the development time period, on top of its other INSuRE obligations. By the week of 25 April, the team had completed both of its required C++ samples.

Team 2 also began sample development in early March. The team started by looking at the necessary Java libraries for implementing the capabilities. The team found that Java File IO, Socket, and ProcessBuilder packages would be useful libraries for development. Throughout March and April, Team 2 developed and debugged the malware samples, dedicating approximately 150 hours toward the development process, and completed both required Java samples by the week of April 25.

Team 1 Ground Truth Sample Characterization

Team 1's final malware sample is programmed in C++. It relied heavily on the Windows API and slightly on C++ standard objects. The team implemented the malware in a modular fashion. The main executable, nicknamed Nikita, is linked with three modules contained in a library, named NikitaFull.lib, at runtime. The Nikita-Full.lib contains the three main functionality modules: a cryptographic operations module named N-Crypt, a file operations module named N-File, and a shell operations module named N-Shell. Given the modular nature of the final malware sample, the main executable simply acts as a base in which the main modules are attached to. It exists to initiate the required connection to the malware controller, which Team 1 dubbed "N-Interface", and manage the connection by processing input from the controller.

Please note that all characterizations made within this section are made from the analyst's point of view when possible. The analysis within the MDLC itself is conducted from the analyst's point of view during the Collect and Analyze phases. Analysis is conducted on materials that the analyst is able to collect, such as stripped malware binaries. These materials have already been subjected to the various OPSEC transforms in the MDLC, as the final C++ sample had been.

Requirements. Of all of the MDLC attacker phases, requirements is the most difficult phase to characterize. At first glance, it may seem simple; one just needs to analyze the requirements. However, to the analyst, these requirements are not concrete. The analyst does not take part in the formulation of requirements between the attackers and any potential clients. In fact, some malware designers may not have a concrete set of requirements. Furthermore, in the case of malware designed for a client, the client may include requirements that are never intended for use and are only included to slow analysis, such as dead code and unused live capabilities. Essentially, analysis of requirements necessitates insider information that is often infeasible to obtain and that was infeasible to simulate in this project. As such, it is impossible to accurately characterize Team 1's final malware sample within the Requirements Phase of the MDLC; no viable data on this end exists. At best, the already-discussed sample requirements characterize this section as well as possible.

Design. First, it is important to view the Design Phase artifacts from a potential malware analyst's position. An analyst's static analysis tools, such as pefile or DetectItEasy (DIE), can easily detect that Team 1's final malware sample is coded in Microsoft Visual C++. Cross-referencing this information with output from a disassembler or decompiler, such as IDAPro, would demonstrate to an analyst that the team's final C++ malware sample was created using a mainly functional paradigm. Furthermore, the analyst would be able to see that the malware's functionality is aided by API function calls, meaning that the malware designer relied heavily on already existing libraries.

The analyst would likely have a difficult time discerning that the malware was modularized using a custom static library. Because Nikita's main executable is compiled with a reference to a custom static library, it is possible that the analyst's toolchain may be unable to detect any calls to the custom static library's code. Static libraries are integrated into an executable at compile-time, meaning that the final executable's disassembly may not reflect that the module code was originally foreign to the main executable. In the scheme of the analysis, this design fact is likely not highly important; an analyst in the Design Phase is more concerned with figuring out how tangible elements of the malware's design will impact the analysis flow and yield hints as to how the malware was engineered.

Independent of the analyst, there are other design features that can be characterized. Team 1 decided to implement its network obfuscation measures using a variable-length exclusive-or (XOR) encryption. Essentially, this entailed embedding an encryption key within the program and using it to perform XOR operations on each byte of network traffic sent between the attacker interface and the implant. Second, the team implemented its anti-debugging measures via timing

checks and Windows API function calls for detecting debuggers. These sort of anti-debugging measures are simple and easily bypassed, but were all that the team had time to include.

Code. It is more useful to view the code phase from the malware designer's point of view; it is simply beyond the scope of this report to explain how an analyst would use a disassembled or decompiled version of the final sample to find code artifacts. However, an analyst could discern almost all of the code-based artifacts by combing through the binary's code in a reverse engineering tool. As such, the Code Phase characterization will be performed by breaking down code decisions for each sample requirement.

Listing Files and Directories. The functionality for list files and directories is implemented within a single function, `listDirs`, within the N-File Module. This function is capable of reading an attacker-supplied directory name from a socket and listing its contents, in terms of files and subdirectories. All of its socket operations are implemented using the `WS2_32` and `MSWSOCK` Windows libraries, both of which provide networking functionality over sockets. For the purposes of all function explanations, the term "sends" implies that the malware is employing the `send` `WS2_32` library function and the term "receives" implies that the malware is utilizing the `recv` `WS2_32` library function.

The `listDirs` function begins by initializing 24 local variables and objects needed to facilitate the directory search request. Next, the function opens the directory using `FindFirstFileA`, which provides an object that can be used to search for all files contained within the directory. After confirming that the directory exists, it searches the directory, with repeated calls to `FindNextFileA`, for all of its contents and organizes the found contents by file type; each file is either a subdirectory or a readable file of some sort. After categorizing the output within two C++ list objects, the `listDirs` function creates a directory report text file with a fixed name and begins iterating through both lists to write all applicable directory information for reporting purposes. Information is gathered through a Windows API structure, `WIN32_FIND_DATAA`, which holds all relevant file information.

Next, the function closes the original report file and reopens it for reading, also using `CreateFileA`. Additionally, the function opens a new file, using the `CreateFileA` API function, to hold the encrypted contents of the report file. At this point, the function encrypts the old file and writes the encrypted data, employing the `WriteFileA` API function, to the new file using a custom encryption function. The encryption function is a custom XOR encryption function that will be discussed momentarily. With the encrypted file in hand, the `listDirs` function sends the file size of the encrypted report and the encrypted file itself to the attacker, which provides OPSEC for the report file transfer. To actually transmit the file, the function employs the `TransmitFile` `MSWSOCK` Windows API function. Lastly, the function deletes the two created files using a call to `DeleteFileA` to eliminate evidence and provide for OPSEC. On the attacker's end, he or she receives the file, decrypts it, and views the directory report.

Uploading Files to the victim. The file upload functionality is implemented within a single function, `uploadSingleFile`, within the N-File Module. The function begins by receiving a file name and the file's size from the attacker. Using the file name and a hardcoded directory

path, the function creates a file path for the file to be uploaded. Next, the `uploadSingleFile` function creates a file using the file path. After this, the `uploadSingleFile` function receives an encrypted version of the file from the attacker in a while loop. For each chunk of the transmitted file, the function decrypts the file chunk and writes it to the new file on the victim machine.

Downloading Files from the victim. The file download functionality is implemented using two subroutines, `extractSingleFile` and `exfilFile`, within the N-File Module. The purpose of the `extractSingleFile` is to verify that the attacker's requested file exists. It begins by receiving a file name from the attacker over a socket. Next, the implant uses `CreateFileA` to open the file, verify its existence, and verify that the received file path is indeed a sendable file. If these conditions are met, the `extractSingleFile` function makes a call to `exfilFile` and provides the function with a handle to the file and the file's path. In `exfilFile`, the file is prepared for a `TransmitFile` operation. First, the function uses `get FileSizeEx` to obtain the file's size and sends the found file size to the attacker over its socket. After this, the function uses `CreateFileA` to create a temporary file to hold an encrypted copy of the requested file. The custom encryption function is then employed to encrypt the file and send it using `TransmitFile`. Lastly, the `exfilFile` function maintains OPSEC by deleting the temporary encrypted file by using `DeleteFileA`.

Execute a file on the victim. The file execution functionality is implemented in a single function, `execMain`, within the N-Shell module. The `execMain` function begins by creating seven local variables. Two of these variables are required Windows API objects for creating processes within the Windows environment: a `PROCESS_INFORMATION` structure and a `STARTUPINFOA` structure. Next, the `execMain` function receives the file path of an executable file and a set of command-line arguments, if applicable, from the attacker. It verifies that the executable exists with a call to `WriteFileA` and tells the attacker whether or not the executable was found. If the executable was found, the function prepares to execute the requested application by allocating the two process structures and settings required values within the structures. Finally, the function calls `CreateProcessA` and executes the program with the specified arguments. If the application successfully executes, the function informs the attacker.

Send commands to the victim. The file execution functionality is implemented in a single function, `shellMain`, within the N-Shell module. This function contains 23 local variables, six of which are required to establish interprocess communication (IPC) and two of which are required to start the command shell process. The other variables served to handle input and output. First, the `shellMain` function uses four `HANDLE` objects and two calls to `CreatePipe` to create two anonymous pipes that the malware can employ to communicate with a spawned command line process. Each of the created pipes are then configured using `SetHandleInformation` API function calls, so that the command line process can inherit and employ the pipes. After setting up the IPC mechanism, the `shellMain` performs similar process setup actions to `execMain`. However, the `STARTUPINFOA` setup is slightly different; it instead maps the process block's standard input, output, and error to the read and write ends of its specific pipes, respectively. Then, the function spawns a `cmd.exe` process using `CreateProcessA`. This process is set to terminate after a single command is received; it was infeasible to send and receive multiple commands at a time.

Once the cmd.exe child process has been created, the function sends an encrypted message to the attacker that confirms that a command can be sent. The shellMain function then receives a command over its socket and processes it. To process the command, the function uses WriteFile to write the received command to the pipe corresponding to the child processes's standard input. Next, the function uses WaitForSingleObjectEx to wait for input to be written to the child processes's standard output or standard error pipe. Once the output has been received, the function employs CreateFileA to create a temporary file to write the command output to. With the file created, the function prepares to read from the anonymous output pipe. This task was complex to implement. First, the function prepares to spawn a thread using the GetCurrentThread, DuplicateHandle, and GetCurrentProcess API functions. The thread routine is simple; it terminates all file IO operations for its parent thread after it is invoked by calling CancelSynchronusIo.

Finally, it is time to read the command output. This operation is performed in an infinite while loop. When the loop is entered, ReadFile is invoked on the output pipe to read its contents. All of the data returned from the read operation is written to the previously created file with the WriteFile API call. The number of read bytes is then considered moving into the next iteration of the loop. Once the loop determines that the next read operation will block infinitely, it creates a thread using CreateThread and enters the next read. Once the read is entered, it is aborted by the thread and the read loop is exited. Finally, the implant sends the command data to the attacker using similar methods to the N-File module functions; GetFileSizeEx and TransmitFile are employed in the exact same method. To provide for OPSEC, the shellMain function uses a TerminateProcess call to kill the child cmd.exe process in the event that a shell error is encountered. This technique ensures that the cmd.exe process does not live past its useful life and potentially trip a resource monitoring program's sensors.

Beaconing banner grabbing. To implement the additional requirements outlined at the end of Phase Two, the team implemented within the main malware implementation to gather system information, named gatherSysInfo. The function gatherSysInfo begins with a call to GetUserNameA, which returns the name of the user who is currently logged in. Next, gatherSysInfo calls GetComputerNameA, which retrieves the name of the system that the implant is installed on. To retrieve the victim's Workgroup or Domain name, gatherSysInfo calls NetGetJoinInformation, which returns the name of the Workgroup or Domain and whether the returned name is a Workgroup or Domain. Based on the returned name type, the function notes the network configuration type. Lastly, the function calls VerifyVersionInfoA to retrieve a OSVERSIONEXA data structure. The function then parses this data structure to determine the operating system's version number, name, and product code. Once the function has gathered all of the requisite information, it constructs a report string, encrypts it, and sends it over its socket to the attacker.

Network traffic obfuscation. The network traffic obfuscation requirement was implemented entirely by employing Windows Cryptographic API function calls and custom functions, housed in N-Crypt, that handled the output of Windows's cryptographic functions. The

network traffic obfuscation implementation begins in the main malware executable and the main malware attacker interface. The two programs, in their startup phases, make a call to `CryptAcquireContext`, which initializes a Windows Cryptographic Service Provider (CSP). Access to a CSP is required in Windows programs to perform secure cryptographic operations. Once access to a CSP is acquired, both functions make a call to a custom function called `deriveCommKey`.

The `deriveCommKey` function takes a handle to the CSP object, a hardcoded passphrase, and any required buffers or objects to handle any input and output as arguments. The function itself is simple. First, the function invokes `CryptCreateHash` to create a hashing object through the CSP. Next, `deriveCommKey` calls `CryptHashData` to create an MD5 hash of the passphrase. Lastly, the function calls `CryptGetHashParam` to write the hashed phrase as a string of bytes to a buffer passed in as an argument. The byte representation of the hardcoded password serves as the key for the encryption function. After `deriveCommKey` returns, the cryptographic setup procedure is finished; both the interface and implant share a key that can be employed to encrypt communications.

The actual encryption of communications is handled by two functions, `encodeNetComms` and `encryptSingleFile`. Both of these functions implement a simple custom algorithm to encrypt network communications and files, respectively. The `encodeNetComms` function is simple and includes zero Windows API function calls. It simply receives a buffer of arbitrary length and the length of the buffer as argument. The function itself consists of a single counted loop, which begins processing the input buffer from the first byte to the specific end of the buffer. The loop itself contains two loops of C code. The first line overwrites the current byte in the buffer with the result of an XOR operation between itself and a corresponding data byte in the hashed password. The hash buffer itself resets every 16 bytes, as this is the output length of an MD5 hash. The second line in the loop simply advanced the byte position in the input buffer. Once the specified number of bytes have been encoded, the loop is exited and the `encodeNetComms` function returns. Once the function returns, the caller can send its newly-encrypted data over its socket. The XOR operation was chosen because it is self-inverting. Calling `encodeNetComms` on an encrypted piece of data decrypts the data. As such, this function also serves as the decryption function for all network communications in the Nikita malware suite.

The `encryptSingleFile` function is the file encryption counterpart to `encodeNetComms`. It takes two file handles as arguments, one for input and one for output. The function is also simple. Within a posttest while loop, the function reads one kilobyte from the input file, encrypts the data using the same custom encryption scheme, and writes the results to the output file. This loop continues until the End of File (EOF) marker is read. Once the loop concludes, the output file should hold the entirety of the input file, but encrypted. Note that the phrases “read” and “write” refer to the Windows API functions `ReadFile` and `WriteFile`, respectively.

Miscellaneous obfuscations. The final malware sample includes relatively few obfuscation techniques beyond the network traffic encryption. The sample uses the Windows API to perform timing-based anti-debugging actions. The first anti-debugging technique consists

of two calls to `GetTickCount64` function, which provides a time code unit in milliseconds that measures the time since the system was booted. The first call acts as a start time to measure the execution time of a fast code section. The second call is placed directly after the code section concludes. If the measured time is far greater than the expected time to execute, then the presence of a debugger can be reasonably assumed and the malware terminates itself. After these two calls, the sample initiates its second anti-debugging technique, which is a call to `IsDebuggerPresent`. This function returns a boolean true value if the malware is found to be running in a debugger. These two techniques combined can defeat simple debuggers and sandbox-based malware analysis tools and antivirus programs.

In addition to the anti-debugging techniques, the final sample also includes various pieces of dead code and unused variables. These artifacts include conditional statements that lead to deleted debugging statements and declaration of variables that are not employed. By leaving these artifacts in the main malware sample, analysis is somewhat slowed because an analyst must analyze various dead ends within the code and must discern the use of variables that are never actually employed. Overall, these artifacts represent a minor OPSEC feature in the sense that they serve to slow down code analysis and increase the useful life of the malware sample.

Build. The build process for Team 1's final malware sample leaves behind no debugging information or symbol tables for an analyst to exploit. From the analyst's perspective, only a stripped malware binary would be present. Using commercially available tools, such as DIE, the analyst could only discern that the tool was compiled using Microsoft's Visual C++ compiler. As such, the set of commands employed to build the malware would represent a major OPSEC code transformation. The team compiled the code using the following technique. First, the team compiled each of the previously discussed functionality modules using the command string `"cl /c /Fo[Module Name.obj] [Module Name.cpp]"`. This produced three compiled, but not linked, module code files. Next, the team used Microsoft's LIB utility to link the three compiled modules into a static library employing the command string `'lib /out:"NikitaFull.lib" N-File.obj N-Shell.obj N-Crypt.obj'`. Lastly, the team compiled the entire implant using the command string `"cl NBS_1_CPP_Main.cpp NikitaFull.lib"`. Notably, the process does not specify any code optimization routines that could affect the final code output. The team's build process is depicted in the figure below.

```

C:\Users\Jacob\Desktop\Bruh>cl /c /Fo"N-Crypt.obj" N-Crypt.cpp
Microsoft (R) C/C++ Optimizing Compiler Version 19.31.31105 for x86
Copyright (C) Microsoft Corporation. All rights reserved.

N-Crypt.cpp

C:\Users\Jacob\Desktop\Bruh>cl /c /Fo"N-File.obj" N-File.cpp
Microsoft (R) C/C++ Optimizing Compiler Version 19.31.31105 for x86
Copyright (C) Microsoft Corporation. All rights reserved.

N-File.cpp

C:\Users\Jacob\Desktop\Bruh>cl /c /Fo"N-Shell.obj" N-Shell.cpp
Microsoft (R) C/C++ Optimizing Compiler Version 19.31.31105 for x86
Copyright (C) Microsoft Corporation. All rights reserved.

N-Shell.cpp

C:\Users\Jacob\Desktop\Bruh>lib /out:"NikitaFull.lib" N-File.obj N-Shell.obj N-Crypt.obj
Microsoft (R) Library Manager Version 14.31.31105.0
Copyright (C) Microsoft Corporation. All rights reserved.

C:\Users\Jacob\Desktop\Bruh>cl NBS_1_CPP_Main.cpp NikitaFull.lib
Microsoft (R) C/C++ Optimizing Compiler Version 19.31.31105 for x86
Copyright (C) Microsoft Corporation. All rights reserved.

NBS_1_CPP_Main.cpp
Microsoft (R) Incremental Linker Version 14.31.31105.0
Copyright (C) Microsoft Corporation. All rights reserved.

/out:NBS_1_CPP_Main.exe
NBS_1_CPP_Main.obj
NikitaFull.lib

C:\Users\Jacob\Desktop\Bruh>

```

Figure 3. The build process for Team 1's final malware sample.

Configure. This sample is static; its functionality cannot be changed once built. As such, any configurations to the sample would have to be made before the build phase. The only true configurations that can be made to the sample would include changing the hardcoded network addresses, directory paths, and password. There are no OPSEC transformations to speak of within this section; it is not a concern to implement OPSEC in non-existent or static configuration choices.

Package. The base version of the final sample is not packaged whatsoever. This fact would be obvious during analysis. However, it is possible to package the malware sample in two main fashions, in a traditional executable packer and in an installer. For example, the sample could have been packed using a previously discussed packer, such as UPX or ASPack. Additionally, the team found that a dropper, also known as an installer, can be employed to install any Windows PE file onto a victim system.

The optimal OPSEC packaging for this malware sample is as follows. First, pack the malware using a traditional executable package. Next, use a resource editing program, such as ResourceHacker, to take the packed malware sample and write it as a resource into the dropper malware. Lastly, deploy the installer to drop the packed malware. This way, the installer drops the malware sample in a packed form, making analysis much more difficult. This technique is a common method by which advanced cybercriminal and nation-state threats deploy malware.

Team 2 Ground Truth Sample Characterization

Team 2's final malware sample is programmed in Java. It relied heavily on Java libraries. The team implemented the malware implant as a single tightly coupled file rather than in a modular fashion.

Requirements. Similar to Team 1's malware sample, the requirements for Team 2's malware sample is difficult to characterize from an analyst perspective. The functional capabilities that would be apparent in the decompiled code, which conform to the previously discussed sample requirements, would be the best characterization of this stage.

Design. An analyst would be able to tell that the implant was designed using the object-oriented programming paradigm by virtue of it being programmed in Java, which is a purely object-oriented language. The sample made extensive use of existing Java packages, which would allow the analyst to see that the functionalities of the sample are designed largely in a modular fashion despite the malware implant consisting of a single file.

Code. It is beyond the scope of this report to explain how an analyst would acquire understanding of code artifacts from the output of a decompiler. It is more useful to view this phase from the malware developer's perspective. The characterization of this phase will be performed by going through the coding decisions made for each sample capability. Overall, the sample makes use of existing Java packages for file I/O, TCP Socket, and interactions with the underlying OS; these packages can be detected by the analyst after the sample binary has been unwrapped and decompiled, which would aid in understanding how these capabilities are handled due to existing knowledge and documentation of these Java libraries.

Remote Connection. The remote connection established between the attacker and victim's machines is created via a TCP Socket connection, with the attacker's machine as the server and the victim's machine as the client. The network addresses are hardcoded into the implant and points to the IP address and port of the controller's machine. This capability is programmed using the java.net package, which include classes such as Socket and ServerSocket that were used in forming the TCP connection. The attacker-side program also makes use of the standard Java Thread class to spawn a separate thread for every new connection for ease of handling exceptions.

Listing Files and Directories. The functionality for listing files and directories given a path as input is implemented using a single method, showDirectory, within the implant. This function accepts a string parameter containing the directory to be listed. It then made use of the java.io.File class to check whether the directory path input by the attacker is a directory that exists on the victim's file system and sends back an error if it is not. Note that despite the class name, objects created through the File class can be either files or directories. If the given directory does in fact exist, the implant iterates through the files and subdirectories within the input directory via the listFile() method, concatenate their names and categories (file/directory) into a string, and return the string to be encrypted (as byte array) and sent through the socket.

Once the encrypted string containing the listed files and subdirectories under the given directory are sent through the socket, the attacker-side program decrypts the received data and prints it to console to inform the attacker of the files and directories on the victim's machine.

Uploading Files to the victim. The functionality for uploading a file onto the victim's machine is implemented using a method, `sendFile`, within the attacker-side program, and a corresponding method in the malware implant. The method accepts a string parameter and an output stream object that outputs data to the socket. It makes use of the `java.io.File` class to check if the file exists, and then made use of the `java.io.FileInputStream` class to read the file byte-wise. It first sends a message to the victim machine indicating that it is sending a file along with the size of the file being sent. The attacker's program then reads the file in chunks of 1KB into byte arrays and encrypts them to send through the socket. The implant on the victim machine receives the data, decrypts the byte chunks, and makes use of the `java.io.FileOutputStream` class to reconstruct the file byte-wise on the victim's machine.

Downloading Files from the victim. The functionality for downloading a file onto the victim's machine is implemented using a method, `recvFile`, within the attacker-side program, and a corresponding method in the malware implant. This functionality parallels the file upload capability, but with the roles swapped. The implant method accepts a string specifying the path to the file to be downloaded and uses the `java.io.File` class to verify its existence. Then it makes use of the `java.io.FileInputStream` class to read the file byte-wise, encrypt, and send through the socket. Finally, the attacker's program receives the data, decrypts, and makes use of the `java.io.FileOutputStream` class to reconstruct the file onto the attacker's machine.

Execute a file on the victim. The functionality for executing a file on the victim's machine is implemented using a method, `executeFile`, within the implant on the victim's machine. The method accepts a string parameter specifying the path to the file to be executed. It makes use of the `java.io.File` class to check if the file exists and is executable. Then it makes use of the `java.lang.Process` and `java.lang.ProcessBuilder` classes to build a process, or execute, the specified file. To read the output of the executing file, the implant uses a `java.io.InputStreamReader` object to catch any output to stdout and store that output in a string to be encrypted and sent through the socket. The attacker's program simply receives the encrypted output, decrypts, and prints the output to inform the attacker of the output of the executed file.

Send commands to the victim. The functionality for executing arbitrary commands on the victim's machine is implemented using a method, `executeCommand`, within the implant on the victim's machine. The method accepts a string parameter specifying the command to be executed. It makes use of the `java.lang.Process` class to create a process that runs the given command using `cmd.exe`, which is the default command line interpreter on Windows machines. Then, similarly to the execute-a-file capability, the implant uses a `java.io.InputStreamReader` object to catch any output to stdout and store that output in a string to be encrypted and sent through the socket. The attacker's program simply receives the encrypted output, decrypts, and prints the output to inform the attacker of the output of the executed command.

Network traffic obfuscation. Network obfuscation is achieved in this sample using XOR encryption of byte data. A custom function was written that takes a provided ASCII string as the encryption key and the byte array containing the data to be encrypted as parameters. The string is hashed using the `java.security.MessageDigest` class into an MD5 hash to be used as an encryption key. The byte representation of the hashed string serves as the key for XOR encryption, and the XOR operation is performed on the provided data in 1 KB blocks.

Miscellaneous obfuscations. This malware sample uses relatively few anti-debugging and anti-analysis obfuscation techniques. It made use of the `java.time` package to measure the time taken to execute between certain points in the program, and if the measured time is greater than the expected time to execute, then the malware terminates itself to protect against an analyst dynamically analyzing the malware.

Build. The build process does not leave behind any debug information for analysts to collect about the sample. The team compiled the sample code using `javac`, the standard Java Programming Language Compiler. The command used to build the class file was “`javac [ImplantName].java`”. This produces a `.class` file that can be run by the Java Virtual Machine, or which can be packaged into a `.jar` file and wrapped into a `.exe` file to run natively.

Configure. The sample is static; its functionality cannot be changed once built. As such, any configurations to the sample would have to be made before the build phase. The network addresses, ports, and directory paths are hardcoded, so they could be considered configurations of the sample. However, they would need to be directly changed in the code before the build phase in order to adapt to each instance of the implant, so they do not characterize the configuration phase. There are no OPSEC transformations within this section regarding this sample.

Package. The malware sample is packaged using IntelliJ IDEA 2021.3.2, Build #IC-213.6777.52, Artifact Builder to create a `.jar` file containing the `.class` file of the implant. Then the `.jar` file was wrapped using the Java launch4 wrapper to create a PE executable, allowing it to be natively run on the victim’s Windows machine.

Summary of Sample MDLC Characterizations

Both samples were developed following the same set of high-level requirements that specified the functionalities as well as OPSEC transformations to be implemented. The characterization of the requirement phase for both samples were found to be the most difficult to acquire from an analyst’s perspective due to the lack of insider information that often is the case for malware analysis. For the design phase, the samples provided by the two teams cover both functional and object-oriented programming paradigms and varying levels of modularity in the implementation of the sample’s capabilities. For the code phase, the samples developed varied in their use of libraries and API functions. Team 1’s sample was developed in C++ and made extensive use of static C libraries and Windows API functions for File IO and Executing files and commands. Team 2’s sample similarly relied on Java libraries for the same functionalities. Both of these characterizations can be acquired by an analyst when viewing the decompiled code

of the sample executables. The OPSEC transformation done in this phase in both teams' samples involved XOR encryption of network traffic, time-based anti-debugging techniques, and deadcode obfuscation. These OPSEC transformations are more difficult from an analyst's perspective to collect and analyze. For the build phase, both teams opted for the typical build tools of their respective programming languages, with Team 1 building the C++ sample using Microsoft's Visual C++ compiler, and Team 2 building the Java sample using the Java Programming Language Compiler. From an analyst's perspective, the compilation process used can be detected from the malware binary by static analysis tools. In both samples, build artifacts such as debugging information and symbol tables are not present in the built binaries for analysts to exploit. For the configure phase, both samples are static in functionalities and cannot be configured after the build process, so there are no configuration options to consider in the characterization of this phase. Finally, for the package phase, Team 1's sample is not packaged, while Team 2's sample is packaged as a .jar file and wrapped into an .exe file using freely available software.

Proposed Malware Analysis Tool

Given that the team did not have the time to implement a malware analysis tool, we instead provide a tool framework that can provide a future team with a blueprint to manufacture a worthy malware analysis tool.

Tool Characteristics. The team proposes that the final tool be programmed within Python. The reason for this is two-fold. First, Python is an easy-to-learn programming language. Many teams will be able to gather skilled programmers for the task of implementing the tool. Second, Python provides an ideal environment to analyze the ground truth samples in. Command line malware analysis tools can be easily executed from the Python interpreter and any applicable output can be easily parsed using either Python's built-in functionalities or third-party parsing libraries. Additionally, a rich toolchain exists for analyzing malware using natively-coded third-party Python tools. The team's toolchain suggestions are discussed later.

Tool Data Structures. There are seven essential data structures for creating this malware analysis tool. First, we propose that six MDLC phase classes are implemented. Each phase class would implement the following data structures: a title string for a particular malware design decision, a description of the choice's significance, a list of tools that were employed to discern that a particular tools were employed and potentially tool output to support the analysis, and, lastly, a boolean flag to represent whether or not a choice is an OPSEC transform. Next, we propose a main MDLC class that implements lists of each MDLC phase class. In essence, this main MDLC class serves as a base class that implements lists of design choices that were identified using all available toolchains for a particular sample. This MDLC-encompassing class would serve as the end product of the analysis. With all of the detected design decisions in hand, the program would print the final decision list in a manner that is human-readable and conducive to aiding the later stages of analysis, such as advanced static analysis employing disassemblers

and decompilers and the dynamic analysis gauntlet that actually requires running and observing the malware sample.

With our recommendation for creating these seven data structures, the team recognizes that the implementation of these classes would be a colossal undertaking. As previously stated, there are dozens of potential design choices that a malware designer can employ for each potential choice during the entirety of the MDLC. As such, there are tens of thousands of choices to categorize. It is simply impossible for a team of four to categorize all of the potential decisions that can be made for producing even basic malware samples such as the samples that our team was required to produce. It is an undertaking for an organization dedicated to advancing the malware analysis field. We make the following observation; it is likely that categorizing the set of malware design choices will require large storage spaces. Each choice would need to be parsed using something similar to a signature that an antivirus program would employ. Essentially, the tool programmers would run a particular tool, analyze the output of the tool, determine toolchain output that characterizes a certain decision, and create a parsing signature that recognizes that signature. This process would need to be performed for every new tool added to the toolchain, which is time-prohibitive for smaller groups. We do, however, discuss suggestions for mitigating this massive workload in the future work section.

Relevant Toolchains. The following list illustrates tools and techniques that we deemed useful for the potential implementation of our malware tool blueprint. The list is organized by the programming language that each tool would be best employed against. This list is by no means a comprehensive list; it only represents the tools that the team was able to locate and deem worthy of inclusion on the list. There are likely many more tools that can prove invaluable to this effort.

- C++ Malware Samples
 - Rizin, Mandiant FLARE-FLOSS, and Mandiant CAPA
- Java Malware Samples
 - Manual Technique: Extracting a .jar file from a Java Virtual Machine wrapper
 - Konloch to decompile the extracted .jar file
- Python Malware Samples
 - pyinstxtractor, pycdump, Python Standard Library modulefinder module

Tool Execution Flow. To characterize the proposed execution flow of our malware analysis tool, the team has devised a flow diagram.

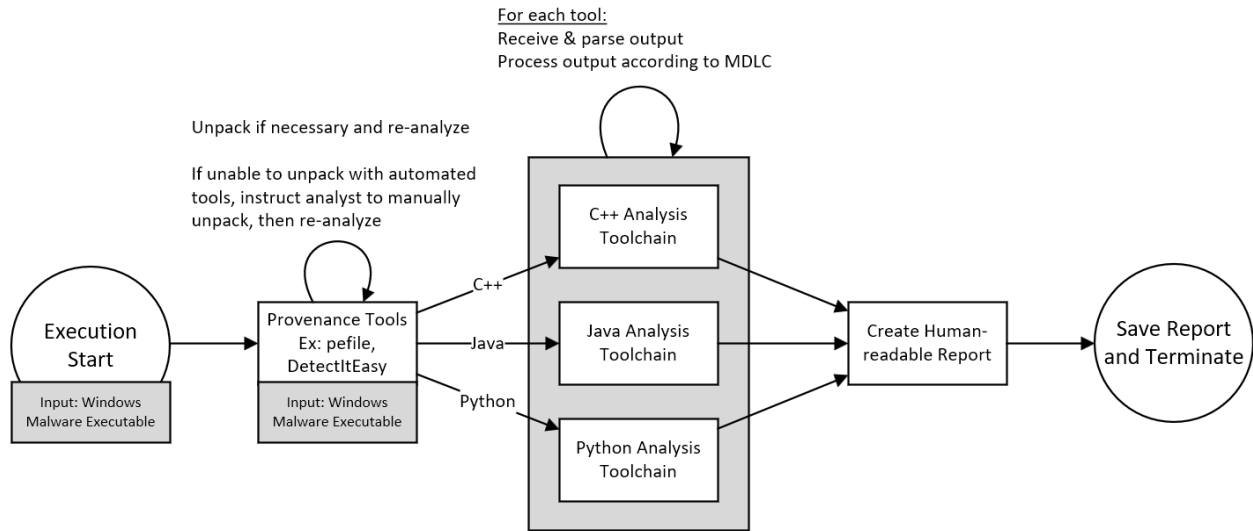


Figure 4. The execution flow diagram for our proposed malware analysis tool.

The proposed tool's execution begins with an analyst specifying a Windows malware executable as input. The input executable is then run through a variety of provenance tools, such as DIE and pefile. These provenance tools serve to classify an executable by its compiler or packer type. If the executable is packed, the tool will attempt to unpack it automatically using available tools. If these attempts are not successful, or the tools for unpacking a certain executable are not integrated into the tool, then the analyst will be directed to unpack the sample manually. Otherwise, if the sample is unpacked by the tool or the file was determined to be unpacked, it is sent for processing by its determined programming language. During the processing step, the sample is repeatedly examined by a toolchain designed to analyze its specific programming language. For each tool in the toolchain, the sample is analyzed by the tool; the tool output is then parsed and sent to a lookup database to find any MDLC characterization data that exists. If the data exists, then any found information artifacts are wrapped into an object of its corresponding MDLC class. All created objects are then added to the main MDLC class for linking onto the phase objects lists within the class. As previously stated, this process repeats for every tool in the toolchain. After this toolchain analysis cycle, a human-readable report is produced. Finally, to conclude the execution of the tool, the report is saved and the tool terminates.

6.0 Issues

Table of Issues

Brief Statement of Problem	Impact	Priority or Rank	Issue Description and Mitigation
Virtual development and testing bed needed	High	1	The team needs a testing bed for our ground truth samples and tool development. Team was challenged with creating its own

			testing bed, and development was significantly slowed due to this. Mitigation: Contact UA Engineering IT Department and Work Together to have a test bed created. Mitigation Failed: The UA engineering IT department was unable to provide the team a test bed. Backup Mitigation of employing a cloud virtualization provider also failed; our project was against most providers' terms of service Team was forced to set up our own virtual machines.
Massive Time Zone Differences	Medium	2	Our team was spread across three different time zones: Arizona Time, Central Standard Time (CST), and India Standard Time (IST) Mitigation Efforts: Set a time frame every team member is available to meet online. Meet in these time frames. Hold additional ad-hoc meetings as needed; organized over Discord Work asynchronously and have each team member check progress if possible. Have team members inform the team of any progress and needs. Mitigation Success: Team typically met regularly. Team relations were strong. Team worked well around the different time zones with minimal meeting issues.
Major Skill Set Differences Between Members	Medium	3	Our team members have vastly different backgrounds. Some members have never been exposed to malware analysis and have little coding experience. Mitigation: Divide the team into two subsections. Have

			subsection leaders assist less experienced members as needed. Where possible, choose coding languages that each team member can understand. The team members with the least experience assigned to research and build their skill set. Mitigation Status: Training on major coding concepts was marginal Samples were complex enough to make code training ineffective in the long-term Deadlines closed too quickly to instill deep coding knowledge
--	--	--	--

The team faced three main issues during the semester. First, the team had severe difficulties in setting up a virtual testing environment. The University of Arizona was unwilling to create a virtual malware testing lab for us; they were concerned about the implications of testing malware on the network. After learning this, the team researched methods by which we could employ cloud services to create a virtual testing environment. However, the team quickly learned that this was against the terms of service of most cloud providers, resulting in the idea being scrapped. As such, the team was forced to create local test beds for each member. This eliminated any possibility of the team collaborating to test our ground truth samples online, which hampered development efficiency. Additionally, finding appropriate testing images and virtualization software took valuable time away from sample development.

Second, the team had major team zone differences. Two members of the team were located in the Arizona Time Zone, one team member was on CST, and another was on IST. The team mitigated this issue by holding meetings only when every member was able to meet. Additionally, the team held impromptu meetings over Discord as needed and worked asynchronously to ensure the project's requirements were met. Overall, these mitigations worked well; team relations were strong and all work was submitted on time.

The final issue was the skill difference between the team members. Only two of the four members had both relevant coding experience in both high and low level languages and any malware analysis or reverse engineering experience. Additionally, the skill gaps in coding and general malware knowledge between the experienced members and the inexperienced members was substantial. The team attempted to mitigate this issue by dividing the team into two subteams for training and work purposes. Additionally, the team attempted to choose coding languages that each member understood. Lastly, the inexperienced team members were assigned to research and improve their coding and malware analysis skills whenever possible. Unfortunately, these mitigations failed. There was not enough time to train the less-skilled group members; there were too many deadlines between INSURE and other obligations to perform any

deep training. The resulting skill differences made it impossible for any language choice to be easy for all group members to understand.

7.0 Conclusions and Recommendations

Ground Truth Sample Quality

Our ground truth samples come from two programming languages C++ and Java. Our C++ ground truth sample meets the minimum requirements with numerous quirks in multiple areas throughout the code while still remaining functional. The C++ sample was able to give directory reading wildcards but being able to execute consecutive commands over shell connections was not possible. The network encoding is weak due to time constraints as implementing a stronger network encryption with the Windows API was significantly more complex and time consuming. Lastly the C++ samples anti-debugging features are elementary at best as it is hard to sync up more effective measures.

The Java ground truth sample was also able to meet all minimum requirements with many of the same elements as the C++ sample. The Java sample also maintained a weak network data encryption for the same reasons as C++ being too complex and time consuming. The Java sample also bears an elementary level anti-debugging feature as again time constraints prevented implementing anything more advanced. All said and done, both samples are functional and are characterizable within the MDLC and serve as acceptable ground truth samples.

Proposed Tool Rationale

Our conclusions and recommendations for a proposed tool rationale and any future work starts by taking a look at how our proposed tool provides a framework for how the MDLC can be examined using commercial-off-the-shelf malware analysis tools. The tool output needs to be handled by the subroutines within the tool while maintaining a human-readable output that describes the attacker's potential thoughts at each stage, as the artifacts are detected.

8.0 Future Work

The team's initial scope for this project was to be able to create a functioning malware analysis tool and was changed on April 15, 2022 at the approval of the team's TD due to time constraints. This was caused due to an underestimation of how long it would take to create and test the ground truth samples. The team was able to complete ground truth samples and give a proposed malware analysis tool, leaving room for another team to take our work and previous teams work to create the malware analysis tool.

Ultimately, the results of our project create major openings for future teams to continue our work; there is much work to be completed. First, each tool handling subroutine would be massive and as there are minute differences in output everywhere that could change how a sample needs to be analyzed each time. A way to mitigate this problem would be to create data stores for each tool and test the tool output versus the known data. We propose that a future team attempts to implement a limited version of our malware analysis tool blueprint. It is suggested

that the team be provided with previous teams' collective ground truth samples. In implementing this tool, the team would analyze the output of a selected toolchain versus the previous teams' ground truth samples. Next, the team would characterize the toolchain's output within the MDLC, create data sets to describe decisions detected by the tool, and implement these data sets within the malware analysis tool's execution flow.

Another major issue is that the tool framework assumes that any given malware is not heavily obfuscated. Any obfuscation will reduce the effectiveness of the proposed tool. A way that this issue can be mitigated would be to identify toolchains that can identify the various forms of obfuscation and proceed to break the obfuscation to ascertain the true choices the malware designer made. The added benefit of finding and employing these toolchains is that they could serve as the chief tools for determining the OPSEC transforms contained within any given sample. Each tool's output could be characterized within the MDLC and turned into a dataset that identifies common obfuscation techniques in order to list OPSEC Code Phase decisions in an understandable manner.

Lastly, the proposed tool's compiler provenance information toolchain is currently too weak. Future teams should attempt to contact experienced research teams to see if they would be willing to provide their advanced compiler provenance determination tools. Obtaining these tools could prove useful for one chief reason. The provenance information could be turned into signature databases that other analysis tools in the toolchain could employ. This could allow a future team to contribute to improving MDLC tool characterization datasets by increasing the number of functions that a specific tool could analyze.

9.0 References

- Andriesse, D., Slowinska, A., & Bos, H. (2017). Compiler-agnostic function detection in binaries. *2017 IEEE European Symposium on Security and Privacy (EuroS&P)*. <https://doi.org/10.1109/eurosp.2017.11>
- Alrabae, S., Debbabi, M., & Wang, L. (2019). On the feasibility of binary authorship characterization. *Digital Investigation*, 28. <https://doi.org/10.1016/j.diin.2019.01.028>
- Alrawi, O., Ike, M., Pruett, M., Kasturi, R. P., Barua, S., Hirani, T., Hill, B., & Saltaformaggio, B. (2021). Forecasting Malware Capabilities From Cyber Attack Memory Images. *30th USENIX Security Symposium*.
- Convert jar to EXE, executable file. Javatips.net. (2020, July 21). Retrieved April 17, 2022, from <https://www.javatips.net/blog/convert-java-jar-file-to-exe>
- Erocarrera. (n.d.). Erocarrera/pefile: Pefile is a python module to read and work with PE (portable executable) files. GitHub. Retrieved April 17, 2022, from <https://github.com/erocarrera/pefile>
- FireEye. (2013, February). *APT1: Exposing one of China's cyber espionage units*. Mandiant. Retrieved February 13, 2022, from <https://www.mandiant.com/resources/apt1-exposing-one-of-chinas-cyber-espionage-units>
- FireEye. (2019). *Double dragon: APT41, a dual espionage and cyber crime operation*. FireEye.

- Retrieved February 13, 2022, from <https://content.fireeye.com/apt-41/rpt-apt41/>
- HexRays. (2011). *Ida F.L.I.R.T. Technology: In-depth*. HexRays Blog. Retrieved February 2, 2022, from https://hex-rays.com/products/ida/tech/flirt/in_depth/
- Konloch. (n.d.). *Konloch/bytecode-viewer: A java 8+ jar & android APK reverse engineering suite*. GitHub. Retrieved April 17, 2022, from <https://github.com/Konloch/bytecode-viewer>
- Mandiant. (n.d.). Mandiant/Capa: The flare team's open-source tool to identify capabilities in executable files. GitHub. Retrieved April 17, 2022, from <https://github.com/mandiant/capa>
- Mandiant. (n.d.). Mandiant/flare-floss: Flare obfuscated string solver - automatically extract obfuscated strings from malware. GitHub. Retrieved April 17, 2022, from <https://github.com/mandiant/flare-floss>
- McVey, S. (2015). Malware Fingerprinting: Analysis of Tool Marks and Other Characteristics of Windows Malware. *NATIONAL CYBERSECURITY INSTITUTE JOURNAL*, 2(3).
- MITRE. (2021). *Mitre ATT&CK®*. MITRE ATT&CK®. Retrieved February 13, 2022, from <https://attack.mitre.org/>
- Posluszny, F. (2020). Malware Similarity Analysis: An adversary perspective. *MITRE Technical Report*.
- Radareorg. (2022). *RADARE2/readme.md at*. GitHub. Retrieved February 13, 2022, from <https://github.com/radareorg/radare2/blob/master/README.md>
- Rahimian, A., Shirani, P., Alrbaee, S., Wang, L., & Debbabi, M. (2015). BinComp: A stratified approach to compiler provenance attribution. *Digital Investigation*, 14. <https://doi.org/10.1016/j.diin.2015.05.015>
- Rizin. (n.d.). *GSOC 2021*. Rizin. Retrieved February 13, 2022, from <https://rizin.re/gsoc/2021/>
- Schirone, R. (2021, December 31). *Rizin shell*. Rizin. Retrieved February 13, 2022, from <https://rizin.re/posts/rzshell/>
- The official Radare2 Book*. The Official Radare2 Book. (n.d.). Retrieved February 13, 2022, from <https://book.rada.re/signatures/signatures.html>

10.0 Team Biographies / Tasking

Timothy Colvert

Timothy Colvert is currently a second-year Engineering management master's student with a Bachelors in Systems engineering from the University of Arizona, with a background in the medical field being an EMT and paramedic for over 5 years. The background as a paramedic has allowed him to understand high stress and quick decision making. He has many classes on learning about people and how they think ranging from learning about different brain level diseases to taking multiple intro level classes on psychology.

He is the lead Systems engineer and IT specialist at Bard Water District working on the development and deployment of a fully automated system to control canal gates and monitor all the water usage. His current job has allowed him to learn a new set of skills from the use of

RTUs and PLCs and the machine coding involved. Learning about radio frequency and connectivity, a wide array of designing wiring diagrams and understanding them, and being able to have a good sense of time management when completing his project milestones. He has general knowledge in a variety of engineering fields due to his degree in systems engineering giving him mechanical engineering, computer and software engineering, low level of coding, and industrial engineering experience.

Mr. Colvert served as the assistant team lead. He was responsible for coordinating the team's meetings and communications, both as they pertain to intra-team dealings and meetings with the technical director.

Abhijeet Dabas

Abhijeet Dabas is a second-year Cybersecurity master's student with a bachelor's degree in Business Administration and an associate degree in Global Operations and Logistics. He has spent over 3 years working in Information Security and Identity and Access management in the banking and insurance industries in Canada. Working in these industries has allowed him to acquire various skills; especially a keen attention to detail necessary to work with confidential and highly sensitive data that could affect the general population and their privacy. Mr. Dabas has also worked on the robotics and process automation team with the goal of automating industry wide functions of a financial institution to mitigate risk of security compromises and information leakage. He has authored over 100 end to end processes and discovery sessions to identify processes for improvement and re-engineering in order to create more efficient and less repetitive workflows to save full-time employee hours for tasks that require attention and creative thinking.

In his first year at the University of Arizona's Eller College of Management, Mr. Dabas worked with machine learning and learned various predictive and descriptive data mining techniques using R language. Additionally, Dabas learned about socket and network programming and architecture. He also gained knowledge about cryptography, experience in TCP/IP networking, and cybersecurity risk analysis.

Mr. Dabas served as one of the principal researchers for the project. Mainly, his research tasks included correlating current threat actors to common malware techniques to determine the most appropriate feature sets to replicate in the teams' malware samples.

Tri Nguyen

Tri Nguyen is an undergraduate Computer Science student at the University of Texas at Dallas. He is also an NSF CyberCorps Scholarship for Service recipient. Mr. Nguyen has completed coursework in C/C++ programming, computer network, and operating system design. As a research assistant, he worked with reported vulnerabilities in open-source C-programs and learned various mitigation techniques for software vulnerabilities concerning memory management, race conditions, and denial of service conditions. During his assistantship, he also developed a module for a Clang-based code parser used in injecting common vulnerabilities into

C programs. He is also competent in Java and Python programming, software engineering, TCP/IP networking, and reverse engineering with GHIDRA. He is currently taking classes in artificial intelligence and database systems.

Mr. Nguyen has extensive programming experience. As such, he was responsible for generating some of the code for the project program.

Jacob Staggs

Jacob Archer Staggs is a three-time Department of Defense CySP scholar under the National Security Agency. Mr. Staggs graduated from the University of Arizona's College of Applied Science and Technology (CAST) with a Bachelor of Applied Science in Cyber Operations Engineering. He is currently a student at the Eller College of Management pursuing a Master of Science in Cybersecurity with an emphasis in physical systems. Throughout his security-related studies, Mr. Staggs was most keenly interested in low-level programming, reverse engineering, malware analysis, malware design, and nation-state geopolitics and intelligence as it relates to the cyber domain. His other course interests included cryptography, traditional networking such as TCP/IP and network topologies, mobile networking including both cellular networks and mobile device protocols, and Python scripting for security applications.

At CAST, Mr. Staggs completed coursework in malware analysis, C programming, Assembly programming, and operating system design. Particularly, he has taken four courses in C programming, to include operating systems programming and C programming for Unix systems, and is competent in memory management, data structures, and manual debugging with GDB. Additionally, Jacob Staggs has experience with both 16-bit Turbo Assembly and 32-bit Netwide Assembly languages, including disassembly with IDAPro and dynamic debugging with OllyDBG. In CAST's Malware Analysis course, Mr. Staggs gained valuable experience in basic static analysis, advanced static analysis, basic dynamic analysis, and advanced dynamic analysis.

Mr. Staggs served as the Team Lead for the project. He was responsible for ensuring that the team is performing properly and that team deadlines are met. He also had extensive programming experience. As such, he was responsible for generating some of the code for the project program.

11.0 Appendix A: Acronyms

ANSI	American National Standards Institute
API	Application Programming Interface
ASCII	American Standard Code for Information Interchange
BAS	Bachelor of Applied Science
BS	Bachelor of Science
C2	Command and Control
CAST	College of Applied Science and Technology
COTS	Commercial Off-the-Shelf
CSP	Cryptographic Service Provider
CST	Central Standard Time
CySP	Cyber Scholarship Program
DoD	Department of Defense
DIE	DetectItEasy
ELF	Executable and Linkable Format
EMT	Emergency Medical Technicians
EOF	End-Of-File
FLIRT	Fast Library Identification and Recognition Technology
GDB	GNU Debugger
HTTP	Hypertext Transfer Protocol
IDAPro	Interactive Disassembler Pro
IO	Input/Output
IPC	Interprocess Communication
IST	Indian Standard Time
IT	Information Technology
KB	KiloByte
MD5	Message-Digest 5
MDLC	Malware Development Life Cycle
MSVC	Microsoft Visual C++ Compiler
MS	Master of Science
NSF	National Science Foundation
OllyDBG	Oleh Yuschuk's Debugger
OPSEC	Operational Security
PE	Portable Executable
PLC	Programmable Logic Controller
RAT	Remote Access Tools
RTU	Remote Terminal Unit
SFS	Scholarship for Services
TCP/IP	Transmission Control Protocol/Internet Protocol
TD	Technical Director
UA	University of Arizona
UPX	Ultimate Packer for eXecutables
UTD	University of Texas at Dallas
XOR	eXclusive-OR

12.0 Appendix B: Terms and Definitions

- **Anonymous Pipe:** An IPC mechanism that can be used to create a flow of data from one process to another
- **Compiler Provenance** - Information contained in a piece of software about the compiler family, compiler version, optimization level, and compiler-related functions.
- **Code Transforms** - Transformations that a code undergoes when it is compiled or interpreted by a code building utility or modified by a packaging tool.
- **Dynamic Analysis** - A malware analysis technique in which a malware sample is run in a controlled environment to gain further understanding of its functionality.
- **Malware Analysis** - The process of examining a malicious object to understand its functions and goal so that it can be identified and classified for future protection.
- **MDLC** - A new malware analysis methodology developed at MITRE that aims to explore the human element of malware development and how it can be exploited to both gain information about malware samples and identify malware developers.
- **Remote Access:** The property of a malware sample relating to its ability to access a system from anywhere over the Internet.
- **Shell:** A tool that provides an attacker a command-line interface to a victim machine.
- **Socket:** A networking data structure that facilitates communications over a network. Consists of an Internet Protocol address, a port number, and various state information maintained by the operating system
- **Static Analysis** - A form of malware analysis in which a malware sample is examined using specific software tools without running the malware itself.
- **Toolmarks** - Any information artifacts that are present in a piece of software, after the build stages, that reveal information about the software itself. For example, compiler provenance information is a subset of software toolmarks.